# AI - Lab

## 4. Transfer Learning



Hochschule Karlsruhe
Technik und Wirtschaft
UNIVERSITY OF APPLIED SCIENCES

Dr. Fabian Sigloch

WS 26/27

# Why transfer learning?

Hochschule Karlsruhe University of Applied Sciences HKA

- Training a CNN from random initial weights usually needs a large and diverse labelled dataset.
- A model trained on a broad task such as ImageNet has already learned useful visual features: edges, textures, shapes, and object parts.
- **Transfer learning** reuses those learned representations for a new target task.

## The basic idea

**large source dataset**

pre-train a model

⇓

**smaller target dataset**

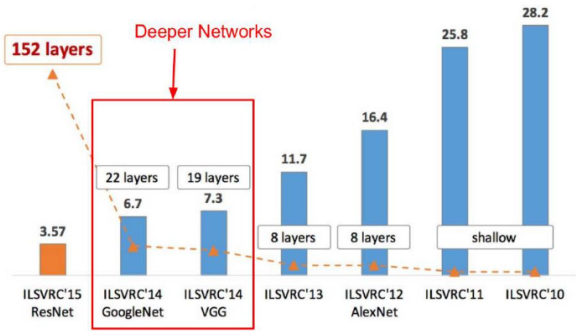
adapt the model to its classes

A common source task is ImageNet classification: around 1.2 million training images from 1,000 categories.

Image: `https://www.image-net.org/`

# ImageNet winners: deeper networks became practical






Increasing depth improved ImageNet performance, but it also made optimisation harder. Residual networks address that problem.

Image: CS231n

# ResNet: learning a residual



- A residual block contains convolution layers and a **skip connection**.
- Rather than learning a complete mapping $H(x)$, the block learns a correction $F(x)$.
- The output is $y = F(x) + x$, followed by an activation.
- The shortcut gives gradients a direct path through a deep network.

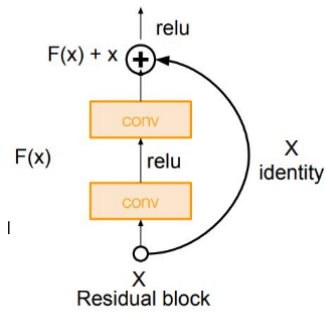



Image: CS231n

# How ResNet builds a hierarchy

- Residual blocks are stacked repeatedly.
- At stage boundaries, spatial resolution is reduced, commonly using stride 2.
- At the same time, the number of channels grows, for example $64 \rightarrow 128 \rightarrow 256$.
- Late layers represent more abstract, task-relevant visual features.

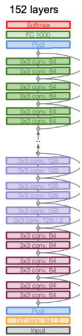
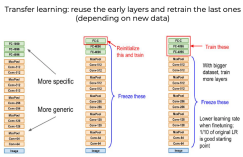



Image: CS231n

# Transfer learning: two main strategies




## Fixed feature extractor

- Freeze the pre-trained backbone.
- Replace and train only the classification head.
- Strong baseline for a small target dataset.

## Fine-tuning

- Initialise from pre-trained weights, then update some or all layers.
- Use a smaller learning rate for existing weights.
- Can adapt features better, but increases cost and overfitting risk.

Image: `CS231n`

# Pre-trained models in `torchvision`

Hochschule Karlsruhe University of Applied Sciences HKA

```python
from torchvision.models import resnet18, ResNet18_Weights

weights = ResNet18_Weights.DEFAULT
model = resnet18(weights=weights)

# Replace the ImageNet head (1,000 outputs)
model.fc = nn.Linear(model.fc.in_features,
                     num_classes)
```

- The downloaded weights contain ImageNet knowledge.
- The final layer must match the classes in the selected dataset.
- `torchvision.models` also provides EfficientNet and many other architectures.

Tutorial: PyTorch Transfer Learning for Computer Vision

# Freeze or fine-tune?

Hochschule Karlsruhe
University of
Applied Sciences
HKA

### Fixed feature extractor

```
for parameter in model.parameters():
    parameter.requires_grad = False

model.fc = nn.Linear(model.fc.in_features,
                     num_classes)
optimizer = optim.Adam(model.fc.parameters(),
                       lr=1e-3)
```

### Fine-tuning

```
for parameter in model.parameters():
    parameter.requires_grad = True

model.fc = nn.Linear(model.fc.in_features,
                     num_classes)
optimizer = optim.Adam(model.parameters(),
                       lr=1e-4)
```

`requires_grad=False` prevents gradient computation and weight updates for the frozen backbone.

# Use the weights' preprocessing

Hochschule Karlsruhe University of Applied Sciences HKA

```
weights = ResNet18_Weights.DEFAULT
preprocess = weights.transforms()

train_data = MyDataset(transform=preprocess)
```

Pre-trained weights expect inputs prepared in a particular way:

- resize and crop dimensions,
- conversion to a tensor,
- normalisation with the statistics used during pre-training.

Using the supplied transform avoids a mismatch between training and transfer-learning inputs.

# Batch normalisation

Hochschule Karlsruhe University of Applied Sciences HKA

- `nn.BatchNorm2d(C)` normalises each of $C$ convolutional channels during training, using statistics from the current mini-batch.
- It then learns a scale $\gamma$ and offset $\beta$:

$$\mathrm{BN}(z) = \gamma \frac{z - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} + \beta$$

- During evaluation, it uses running statistics collected during training.

**Typical CNN block**

`Conv2d` $\rightarrow$ `BatchNorm2d` $\rightarrow$ `ReLU` $\rightarrow$ `MaxPool2d`

Batch normalisation often makes optimisation more stable and allows more robust learning-rate choices.

# Assignment 4: choose and inspect a dataset

Hochschule Karlsruhe University of Applied Sciences HKA

1. Browse the torchvision datasets and choose one classification dataset.
2. Do **not** use CIFAR, ImageNet, FashionMNIST, or EMNIST.
3. In the notebook, display representative example images and state the training and test set sizes.
4. Identify the number of classes, input channels, image dimensions, and the evaluation metric.

## Keep the experiment comparable

Use one defined train/test split and the same evaluation procedure for every approach in the assignment.

# Assignment 4: train a CNN from scratch

Hochschule Karlsruhe University of Applied Sciences HKA

1. Design a CNN similar in spirit to Week 3.
2. Include batch-normalisation layers in the convolutional part of the model.
3. Train it on the training data and measure accuracy on held-out test data.
4. Record the training time and retain the learning curves where helpful for interpretation.

## A useful baseline

This custom CNN is the reference point: transfer learning is worthwhile only if it improves the final result or achieves a comparable result more efficiently.

# Assignment 4: transfer-learning experiments

Hochschule Karlsruhe
University of
Applied Sciences
HKA

1. Use a pre-trained **ResNet18** as a fixed feature extractor.
2. Fine-tune the pre-trained ResNet18; adapt the learning rate appropriately.
3. Repeat both variants with pre-trained **EfficientNet_B5**.
4. Compare all approaches on the same test data: test accuracy and training time are required.

| **Custom CNN** | **ResNet18** | **EfficientNet_B5** |
|---|---|---|
| train from scratch | frozen / fine-tuned | frozen / fine-tuned |

# Interpreting the comparison

Hochschule Karlsruhe University of Applied Sciences HKA

- Report a compact table with model, strategy, trainable parameters, training time, and test accuracy.
- The best accuracy is not the only criterion: a large model may be substantially slower or overfit a small dataset.
- If fine-tuning performs worse, check the learning rate, number of epochs, data augmentation, and whether the pre-trained preprocessing was used.
- For a sound comparison, keep the data split, metric, hardware, and timing scope consistent.

# Submission guidelines

Hochschule Karlsruhe University of Applied Sciences HKA

- Submit one solution notebook (`.ipynb`) per team by e-mail.
- Subject: `[KI-Lab] Assignment 4 - <teamname>`
- Deadline: next Thursday, 23:59.
- The notebook should run top-to-bottom and make the experiment setup and comparison understandable.

# Final words

Hochschule Karlsruhe
University of
Applied Sciences
HKA

- You can start working on Assignment 4 now.
- Decide with your partner how to organise the work.
- I will be available until the end of the slot for questions.
- Questions are welcome in the chat, but please do not post complete solutions.

Happy coding!